universitas
MALIKUSSALEH

Translasi Logika ke Sintaksis

Menjembatani Algoritma Konseptual Menjadi Baris Kode

Ir. Shaki Saptiadi Putra, S.T., M.Eng., IPP.

Program Studi Teknik Mesin
Fakultas Teknik
Universitas Malikussaleh

Agenda Perkuliahan

Jembatan Logika ke Mesin

Variabel & Tipe Data Keteknikan

Translasi Logika Percabangan (If-Else)

Translasi Logika Perulangan (Looping)

Studi Kasus Translasional

Seni Menerjemahkan Pikiran

Sebelumnya kita telah belajar bagaimana menyusun urutan pemecahan masalah menggunakan **Flowchart** dan **Pseudocode**. Namun, itu semua baru sebatas "konsep" di atas kertas.

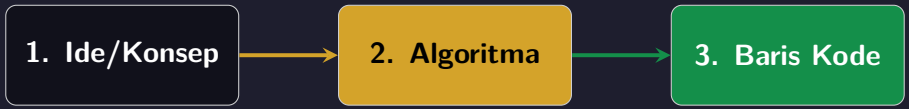
Definisi Translasi Sintaksis

Proses pemetaan (mapping) dari algoritma konseptual yang bebas bahasa, ke dalam aturan tata bahasa (sintaksis) yang kaku dan spesifik milik bahasa pemrograman tertentu (seperti Python, C++, atau MATLAB).

Ibarat seorang insinyur yang telah merancang mesin di *software CAD* (Flowchart), kini saatnya memasukkan parameter tersebut ke mesin *CNC* (Sintaksis) agar benda nyatanya bisa diproduksi.

Alur Transisi Pemrograman

Dalam praktiknya, kita tidak pernah langsung mengetik kode tanpa memikirkan kerangkanya terlebih dahulu.

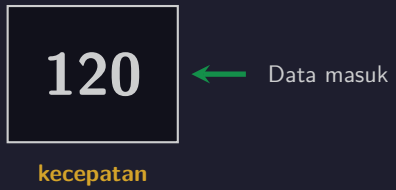


Konsep Variabel: "Wadah Memori"

Dalam memori komputer, **Variabel** adalah sebuah kotak atau wadah yang diberi label nama, yang berfungsi untuk menyimpan data yang nilainya dapat berubah-ubah.

Kaidah Deklarasi (Assignment): Proses mengisi nilai ke dalam variabel selalu menggunakan operator sama dengan (=), di mana nilai di sebelah KANAN dimasukkan ke wadah di sebelah KIRI.

Pseudocode: `SET kecepatan = 120`
Python: `kecepatan = 120`



Pemetaan Tipe Data Dasar

Mesin komputer harus tahu ukuran dan bentuk data (angka utuh, desimal, atau teks) agar tidak boros memori.

Tipe Data	Representasi (Python)	Contoh di Teknik Mesin
Integer	Angka bulat (`int`)	Jumlah gigi roda gigi (Misal: 45)
Float	Angka desimal (`float`)	Suhu ruang bakar (210.5°C)
String	Teks karakter (`str`)	Kode material (Misal: "AISI_1045")
Boolean	Logika Biner (`bool`)	Status saklar/katup (`True` / `False`)

Percabangan: Pengambilan Keputusan

Program yang statis akan mengeksekusi instruksi dari atas ke bawah tanpa syarat. Namun, mesin cerdas memerlukan **Kondisional (Percabangan)**.

Percabangan memungkinkan komputer mengevaluasi sebuah kondisi (menggunakan operator logika seperti >, <, ==) dan memilih jalur eksekusi yang berbeda berdasarkan hasil evaluasi tersebut (*True* atau *False*).

Aturan Emas Evaluasi

Pernyataan kondisi harus selalu dapat dievaluasi menjadi nilai mutlak **BENAR (True)** atau **SALAH (False)**. Tidak ada ambiguitas bagi prosesor.

Komparasi Sintaksis: *If-Else*

Perhatikan bagaimana logika yang sama ditulis dalam gaya yang berbeda saat ditranslasikan. Python mengandalkan indentasi, sedangkan C++ menggunakan kurung kurawal {}.

Bentuk Pseudocode:

```
IF tekanan > 100 THEN
  PRINT "Buka Katup"
ELSE
  PRINT "Aman"
END IF
```

Translasi Python:

```
if tekanan > 100:
  print("Buka Katup")
else:
  print("Aman")
```

Catatan: Titik dua (:) di Python menggantikan kata THEN, dan indentasi ke dalam menggantikan fungsi blok END IF.

Perulangan (Iterasi): Tenaga Kuda Komputer

Komputer diciptakan untuk melakukan perhitungan berulang-ulang tanpa rasa lelah. Di sinilah letak kekuatan utama struktur **Looping (Perulangan)**.

Kapan kita menggunakan perulangan di keteknikan?

- Saat mikrokontroler harus membaca data sensor suhu setiap 10 milidetik tiada henti.
- Saat simulasi metode numerik (seperti Elemen Hingga) menghitung titik matriks matriks satu per satu hingga ribuan kali.

Ada dua jenis utama: **For Loop** (Terukur) dan **While Loop** (Berbasis Kondisi).

For Loop: Iterasi dengan Batas Pasti

Digunakan ketika kita sudah **tahu persis berapa kali** blok kode tersebut harus dijalankan.

Contoh Kasus: Menguji 5 buah silinder berurutan secara otomatis.

```
 # Syntax Python untuk pengulangan 5 kali (0 sampai 4)
for i in range(5):
    print(f"Menguji silinder ke-{i+1}")
    aktifkan_injektor(i)
```

Sintaks `range(5)` akan menghasilkan rentang nilai 0, 1, 2, 3, 4. Iterasi selalu mulai dari 0 pada dunia komputer (Zero-based indexing).

While Loop: Iterasi Berbasis Kondisi

Digunakan ketika kita **tidak tahu kapan perulangan akan berhenti**, tapi kita punya parameter kondisinya. Selama kondisi *True*, perulangan jalan terus.

Contoh Kasus: Memanaskan cairan hingga mencapai target $100°C$.

```
 suhu = 30 # Nilai awal
while suhu < 100:
    nyalakan_pemanas()
    suhu = baca_sensor() # Perbarui variabel kontrol
    print("Masih memanaskan...")
print("Target tercapai, matikan sistem!")
```

Studi Kasus: Monitoring Rotasi Mesin CNC

Mari kita rangkum semua komponen (Variabel, Perulangan, Percabangan) ke dalam satu permasalahan terintegrasi.

Skenario Masalah (Problem Statement)

Sebuah poros *spindle* mesin CNC memantau rotasi (RPM) selama 5 siklus berurutan. Pada setiap siklus, sistem mengevaluasi:

- Jika RPM lebih dari 5000, sistem memberikan peringatan "RPM Berlebih!".
- Jika RPM normal, sistem menampilkan status normal.

Langkah 1: Merancang Algoritma (Pseudocode)

Sebelum menyentuh bahasa pemrograman, kita susun kerangka berpikir logisnya terlebih dahulu:

```
START
   FOR siklus = 1 TO 5 DO
      READ nilai_rpm dari sensor
      IF nilai_rpm > 5000 THEN
         PRINT "Bahaya:  RPM Berlebih!"
      ELSE
         PRINT "Sistem Normal"
      END IF
   END FOR
END
```

Langkah 2: Translasi Final (Implementasi Python)

Sekarang, kita translasikan ke dalam bahasa mesin riil (Sintaksis Python):

```
 # Perulangan FOR sebanyak 5 kali
for siklus in range(1, 6):
    nilai_rpm = get_sensor_data() # Simulasi baca sensor
    # Evaluasi percabangan logika
    if nilai_rpm > 5000:
        print(f"Siklus {siklus}:  Bahaya!  RPM Berlebih!")
    else:
        print(f"Siklus {siklus}:  Sistem Normal")
```

Perhatikan keindahan dan kerapian blok (indentasi) di Python yang mewakili level prioritas eksekusi secara visual.

Kesimpulan: *Mindset Programmer*

Proses translasi ini mengajarkan satu prinsip fundamental dalam rekayasa komputasi:

Logika Bersifat Abadi, Sintaks Bersifat Sementara

Jika Anda menguasai **logika penyelesaian masalah (Algoritma)**, mempelajari bahasa pemrograman baru (Python, C++, Java, atau MATLAB) hanyalah soal menghafal aturan kamus (Sintaksis).

Jangan terjebak menghafal baris kode, tapi asahlah kemampuan memecah masalah besar menjadi urutan logika fisis yang kecil dan berurutan.

Terima Kasih

Ada Pertanyaan?



universitas
MALIKUSSALEH